

Chapter 5: Segmentation

Assoc. Prof. Dr.-Ing. Nguyễn Xuân Hà

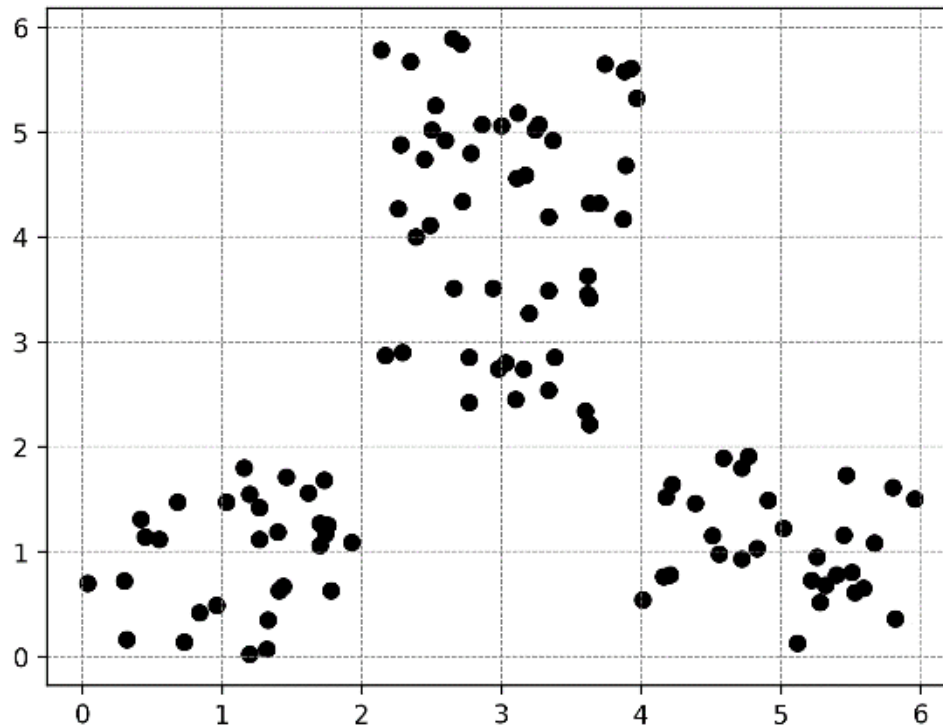
Robotics Group, Dept. of Mechatronics, SME - HUST

ha.nguyenxuan@hust.edu.vn

<https://sites.google.com/site/tsnguyenxuanhabkhn/home>

Lecture 5: Programming exercise

Bài 1: Cho ảnh như hình dưới đây (file 'C5_B1.png' trong thư mục data đi kèm).



Sử dụng phân cụm K-means để phân chia bộ dữ liệu này thành 3 cụm. Chú ý sử dụng hàm `np.loadtxt()` để đọc file này.

Lecture 5: Programming exercise

Thuật toán phân cụm K-means:

- **Bước 1**: Khởi tạo tâm của các cụm dữ liệu $c_1, \dots, c_K$
- **Bước 2**: Xác định dữ liệu cho từng cụm
 - Với mỗi điểm dữ liệu p_i , tìm tâm gần nhất c_j
 - Đặt điểm dữ liệu p_i này vào cụm thứ j
- **Bước 3**: Ước lượng lại tâm của cụm dữ liệu

Tính toán lại điểm tâm cụm c_i của bộ dữ liệu thứ i
- **Bước 4**: Nếu tâm cụm dữ liệu thay đổi, lặp lại bước 2

Lecture 5: Programming exercise

```
data_file = './data/C5_B1.txt'
```

```
data = np.loadtxt(data_file)
```

```
[[1.41  0.63]  
 [0.84  0.42]  
 [1.7   1.27]  
 ...  
 [2.39  4.   ]]
```



Lecture 5: Programming exercise

`K = 3` -----> Số cụm dữ liệu

`center_points = np.zeros((K, 2))` -----> Khởi tạo dữ liệu cho các tâm cụm

`for i in range(K):`

`ci = data[np.random.randint(0, data.shape[0]), :]`

`center_points[i, :] = ci`

-----> Lấy một điểm bất kì trong bộ dữ liệu để làm tâm cụm

Lecture 5: Programming exercise

```
for i in range(max_iter):
```

```
    points_list = []
```

```
    for j in range(K):
```

```
        points_list.append([])
```

Khởi tạo số liệu
cho từng cụm

```
    for j in range(data.shape[0]):
```

```
        data_j = data[j, :]
```

```
        dist = np.zeros(K)
```

```
        for n in range(K):
```

```
            dist[n] = np.linalg.norm(data_j - center_points[n])
```

```
        loc = np.argmin(dist)
```

```
        points_list[loc].append(data_j)
```

Tính khoảng
cách giữa từng
điểm dữ liệu đến
các tâm này

Gán điểm cho
tâm gần nhất

Lecture 5: Programming exercise

```
for i in range(max_iter):
```

```
...
```

```
center_new = center_points.copy()
```

```
for j in range(K):
```

```
    points_j = np.array(points_list[j])
```

```
    center_j = np.mean(points_j, axis=0)
```

```
    center_new[j, :] = center_j
```

Tính tâm mới
bằng trung bình
của bộ dữ liệu

```
check_dist = np.linalg.norm(center_new-center_points, axis=1)
```

Tính khoảng
cách giữa tâm
mới và tâm cũ

```
if np.max(check_dist) > 1e-3:
```

```
    center_points = center_new.copy()
```

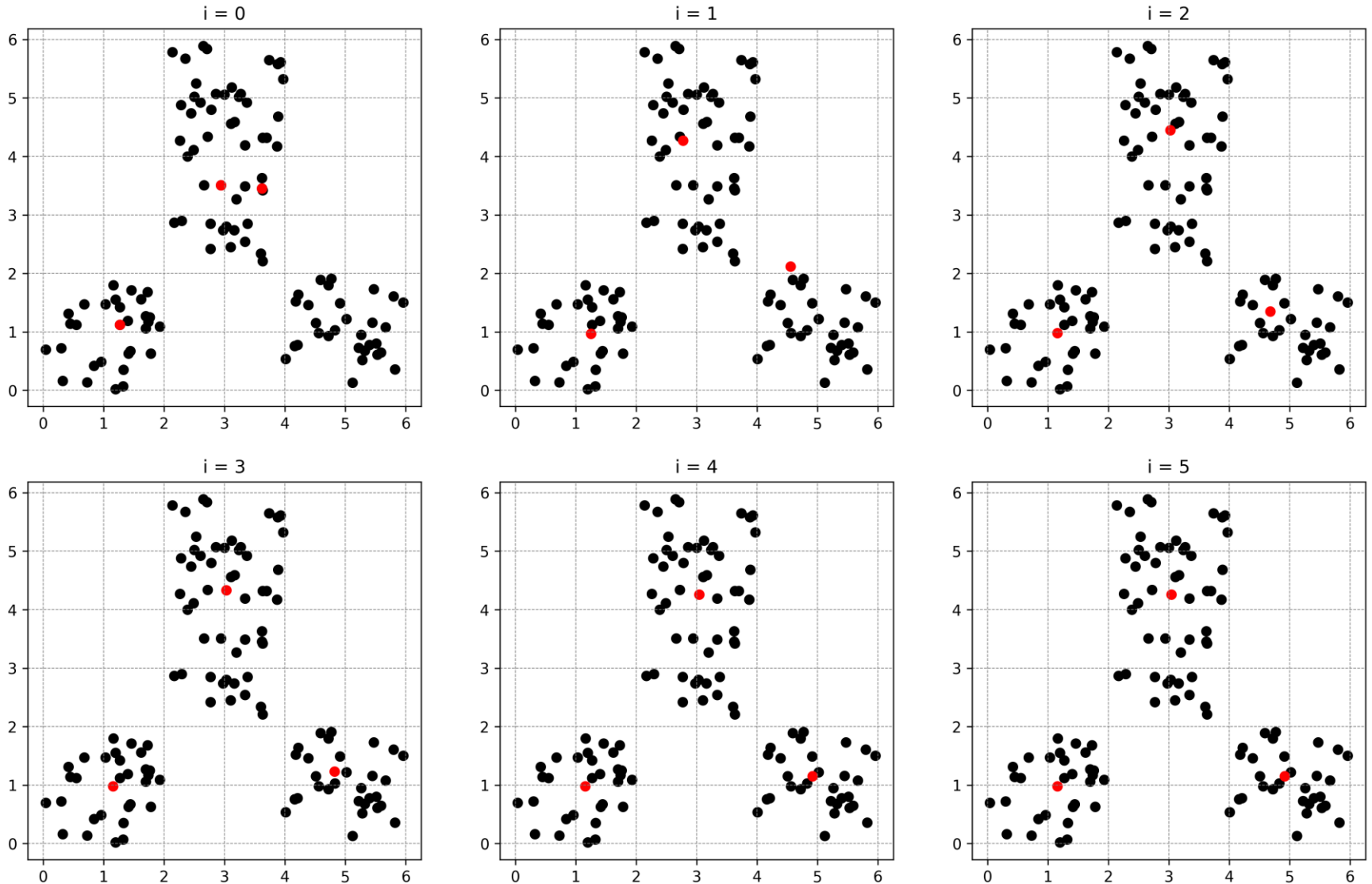
```
else:
```

```
    break
```

Kiểm tra các khoảng
cách này, nếu đủ nhỏ
thì có dừng vòng lặp

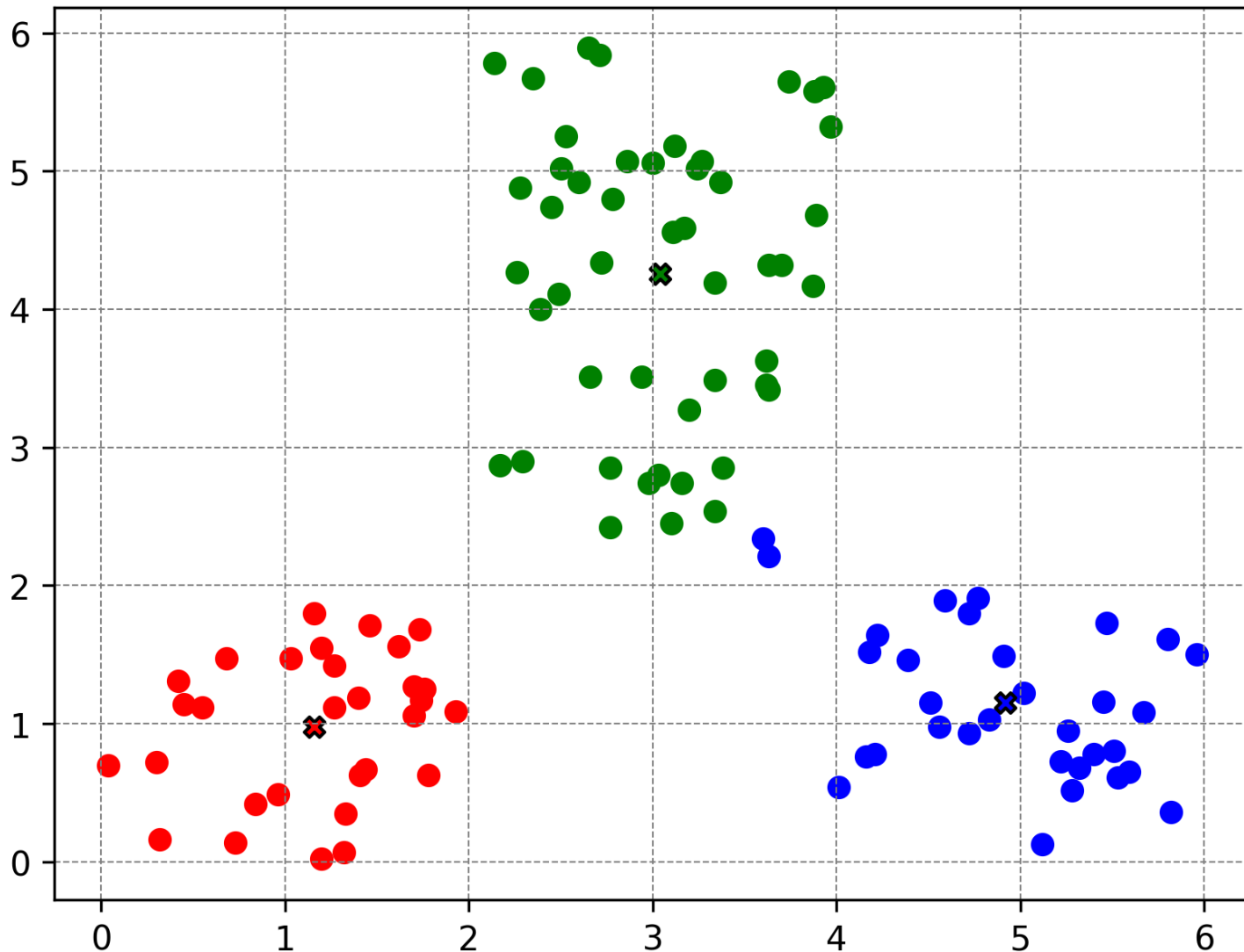
Lecture 5: Programming exercise

Kết quả



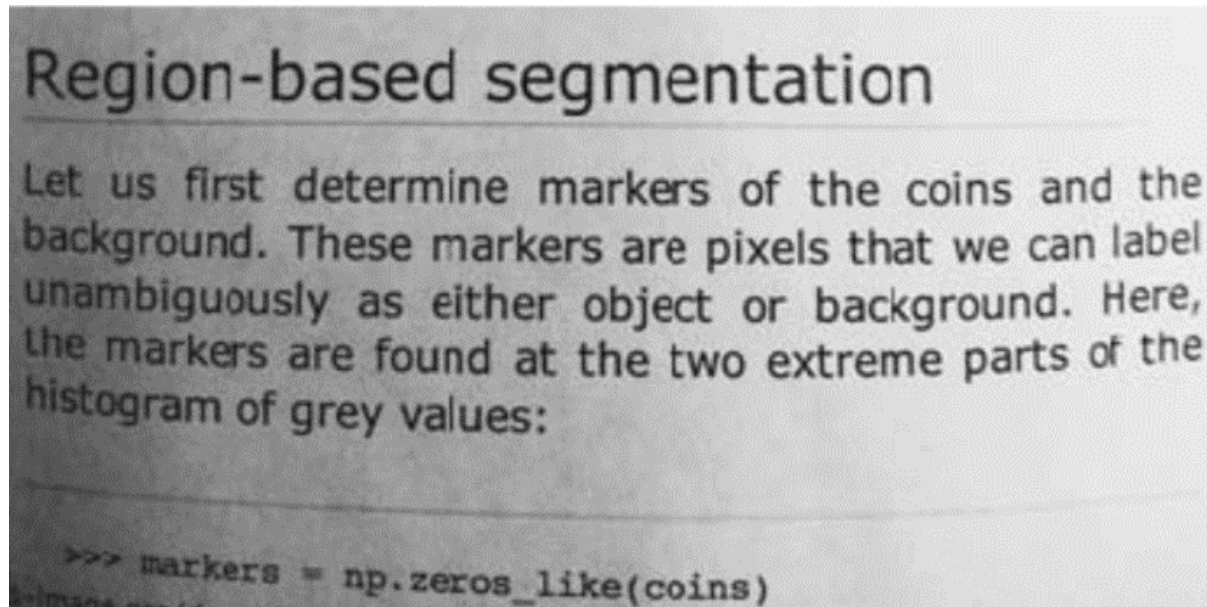
Lecture 5: Programming exercise

Kết quả



Lecture 5: Programming exercise

Bài 2: Cho ảnh như hình dưới đây (ảnh 'C5_B2.png' trong thư mục data đi kèm).



Hãy tiến hành phân ngưỡng cho ảnh trên thành ảnh nhị phân sử dụng phân cụm K-means dựa trên biểu đồ tần số ảnh histogram.

Gợi ý: sử dụng bộ dữ liệu hai chiều gồm: độ xám – tần số để tiến hành phân chia.

Lecture 5: Programming exercise

```
img1 = img_gray.copy()
```

```
threshold = 150
```

```
img1[img_gray < threshold] = 0
```

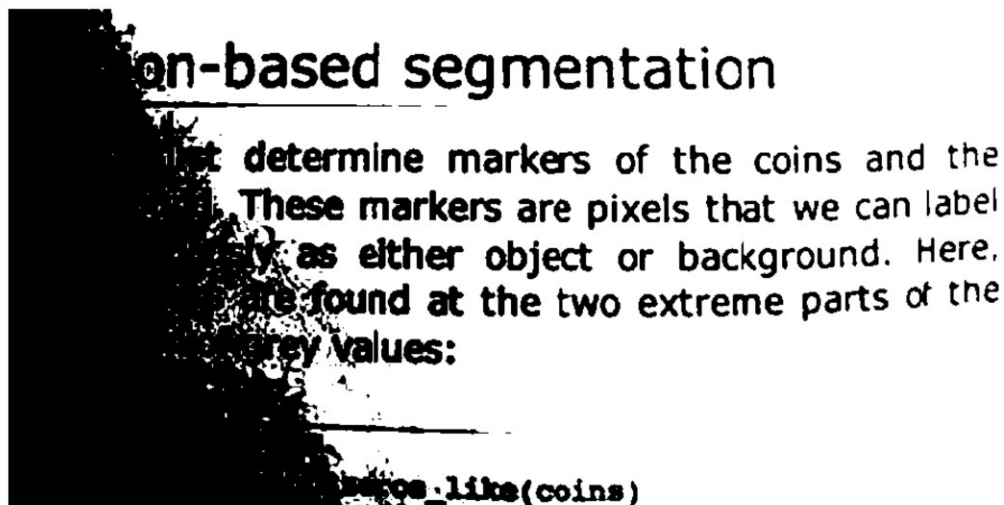
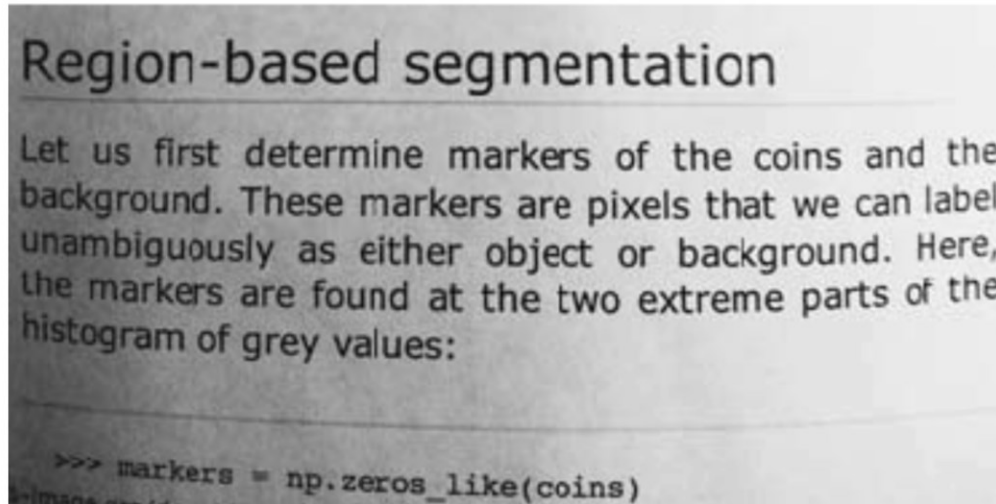
-----> Đưa các pixel có độ xám thấp hơn ngưỡng về mức 0

```
img1[img_gray >= threshold] = 255
```

-----> Đưa các pixel có độ xám cao hơn ngưỡng về 255

Lecture 5: Programming exercise

Kết quả phân ngưỡng ảnh xám với ngưỡng cố định



Lecture 5: Programming exercise

```
def GetHistogram(img):
```

```
    h = np.zeros(256)
```

```
    for i in range(img.shape[0]):
```

```
        for j in range(img.shape[1]):
```

```
            val = img[i, j]
```

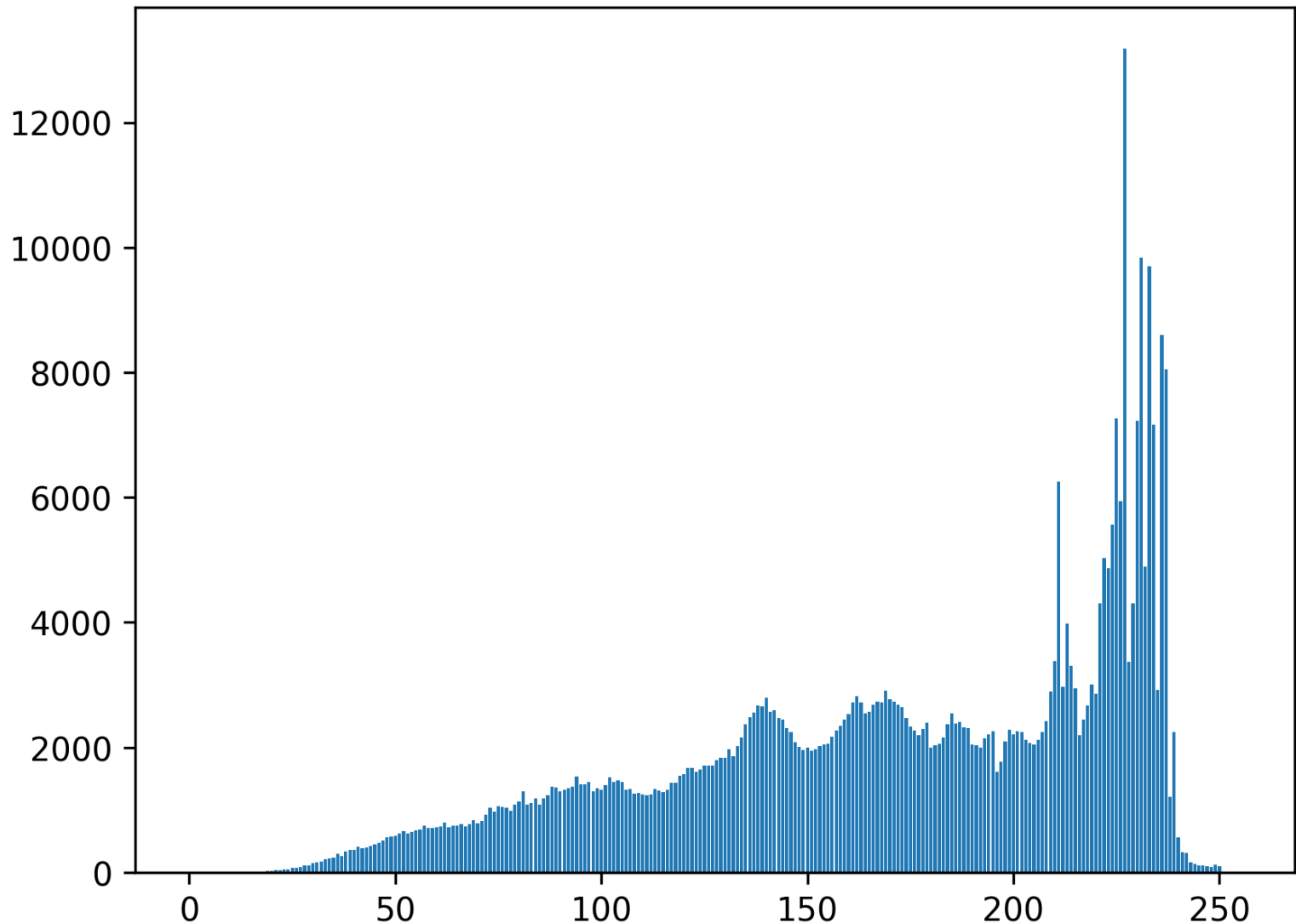
```
            h[val] += 1
```

```
    return h
```

Tính tần số xuất
hiện cho từng
mức độ xám

Lecture 5: Programming exercise

Biểu đồ tần số ảnh Histogram



Lecture 5: Programming exercise

```
data = np.zeros((256, 2))  
data[:, 0] = np.arange(0, 256)  
data[:, 1] = h
```



Khởi tạo dữ liệu
Cột 1: mức xám
Cột 2: Tần số tương ứng

```
c1 = data[np.random.randint(0, 256), :]  
c2 = data[np.random.randint(0, 256), :]
```



Lấy một điểm bất
kì trong bộ dữ liệu
để làm tâm cụm

Lecture 5: Programming exercise

```
if c1[0] > c2[0]:  
    c1, c2 = c2, c1  
    c1_list, c2_list = c2_list, c1_list
```

Đảo lại thứ tự cụm dữ liệu để đảm bảo cụm 1 tương ứng với màu đen, cụm 2 tương ứng với màu trắng

```
img2 = img_gray.copy()
```

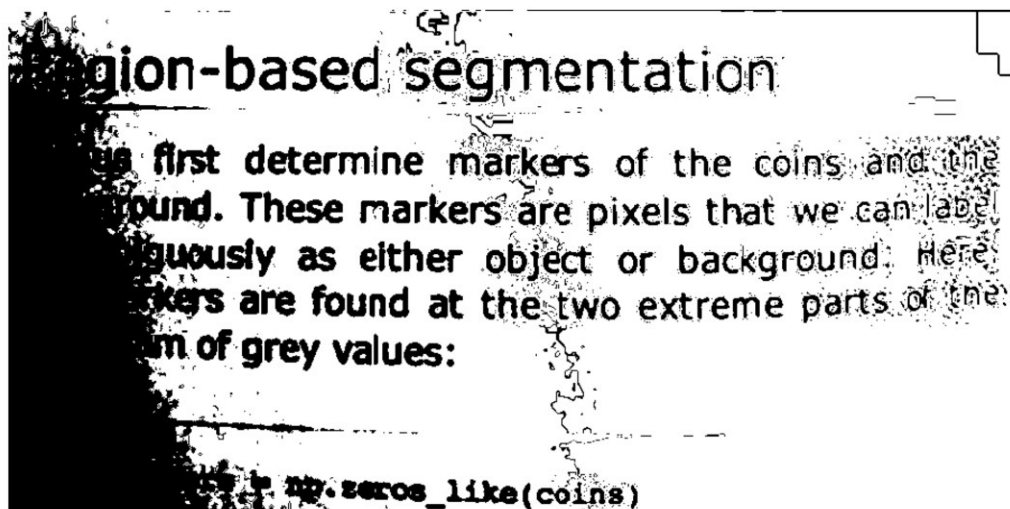
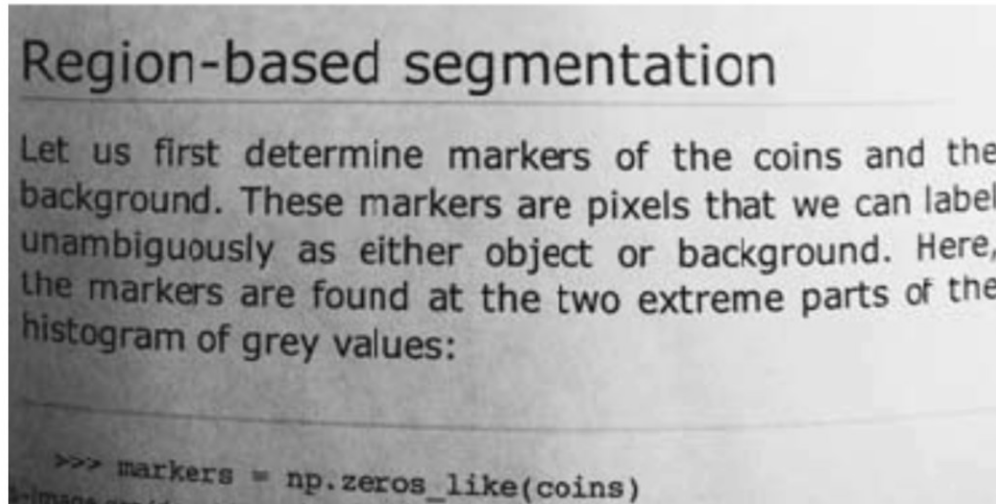
```
for i in range(c1_list.shape[0]):  
    img2[img_gray == c1_list[i]] = 0
```

```
for i in range(c2_list.shape[0]):  
    img2[img_gray == c2_list[i]] = 255
```

Tiến hành phân mảnh ảnh dựa trên các dữ liệu tương ứng với từng cụm

Lecture 5: Programming exercise

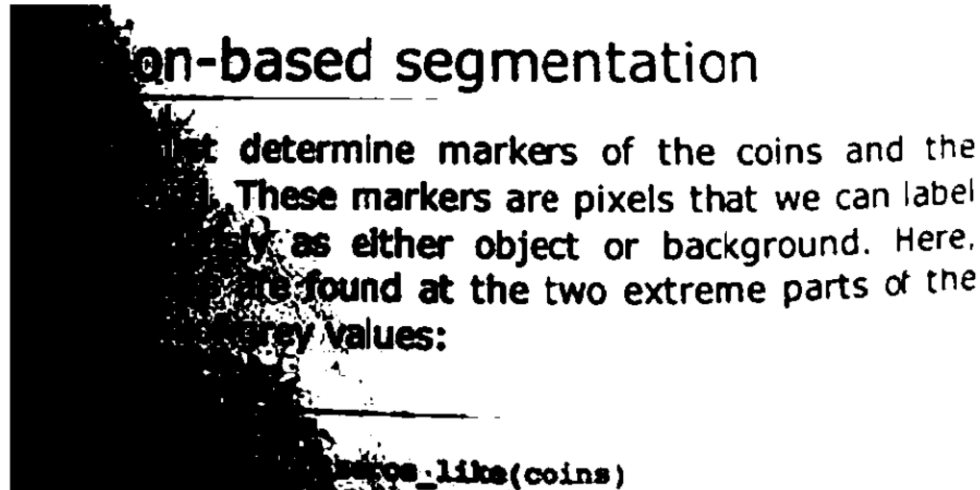
Kết quả phân cụm K-means



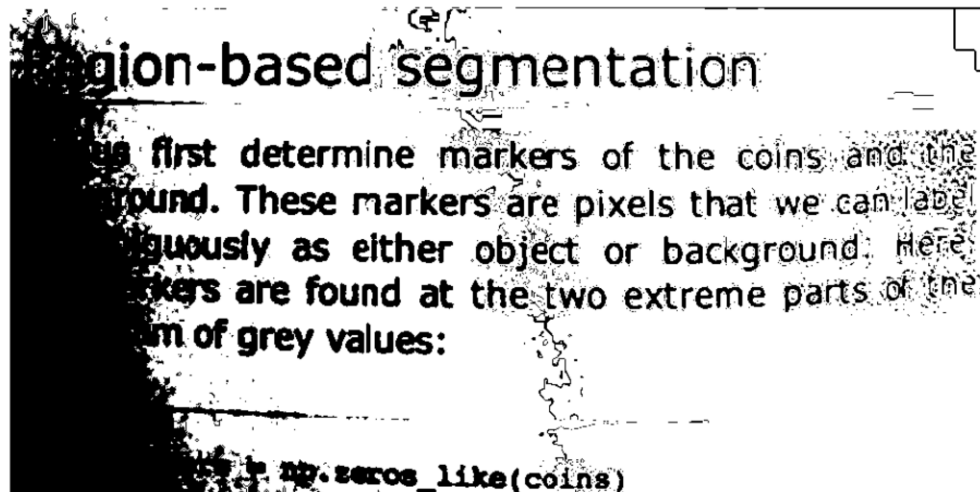
Lecture 5: Programming exercise

So sánh phân ngưỡng cố định với phân cụm K-means

Constant Threshold



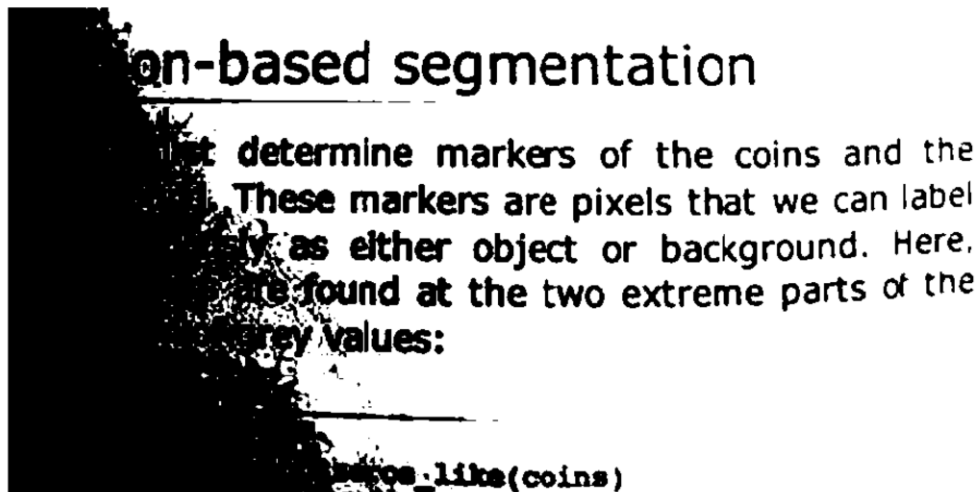
K-means



Lecture 5: Programming exercise

Có cách nào tăng chất lượng phân mảnh ảnh không?

Constant Threshold



K-means

Region-based segmentation

Let us first determine markers of the coins and the background. These markers are pixels that we can label unambiguously as either object or background. Here, the markers are found at the two extreme parts of the histogram of grey values:



Lecture 5: Programming exercise

Bài 3: Cho ảnh như hình dưới đây (ảnh 'C5_B3.png' trong thư mục data đi kèm).



Hãy tiến hành phân chia ảnh trên sử dụng K-means với số cụm bằng 5

Lecture 5: Programming exercise

```
img = cv2.imread('./data/C5_B3.png')
```

```
data = np.zeros((img.shape[0] * img.shape[1], 3),  
                dtype=np.uint8)
```

```
data1 = np.zeros((img.shape[0] * img.shape[1], 2),  
                 dtype=np.int64)
```

```
for i in range(img.shape[0]):  
    for j in range(img.shape[1]):  
        data[i*img.shape[1] + j, :] = img[i, j, :]  
  
        data1[i*img.shape[1] + j, :] = [i, j]
```

→ Khởi tạo dữ liệu

- **data** chứa dữ liệu về giá trị của các kênh
- **data1** chứa vị trí tương ứng của các giá trị bên trên

Lecture 5: Programming exercise

```
K = 5

max_iter = 30

center_points = np.zeros((K, 3))

for i in range(K):
    center_points[i, :] = data[np.random.randint(0, data.shape[0]), :]
```



Khởi tạo tâm cụm

Lecture 5: Programming exercise

```
for i in range(max_iter):
```

```
    points_list = []  
    for j in range(K):  
        points_list.append([])
```

Khởi tạo số liệu
cho từng cụm

```
    dist = distance.cdist(data, center_points)
```

Tính khoảng cách
giữa từng điểm dữ
liệu đến các tâm này

```
    loc = np.argmin(dist, axis=1)
```

```
    for j in range(data.shape[0]):  
        points_list[loc[j]].append(j)
```

Gán điểm cho
tâm gần nhất

Cần sử dụng thư viện này để tính khoảng cách:

```
from scipy.spatial import distance
```

Lecture 5: Programming exercise

```
for i in range(max_iter):
```

```
    ...
```

```
    center_new = center_points.copy()
```

```
    for j in range(K):
```

```
        center_j = np.mean(data[points_list[j], :], axis=0)
```

```
        center_new[j, :] = center_j
```

```
    check_dist = np.linalg.norm(center_new - center_points,  
                                axis=1)
```

```
    if np.max(check_dist) > 1:
```

```
        center_points = center_new.copy()
```

```
    else:
```

```
        break
```

Tính tâm dữ liệu
cho cụm dữ liệu

Kiểm tra các tâm
dữ liệu xem có sự
thay đổi không

Lecture 5: Programming exercise

```
img2 = img.copy()
color1 = [0, 0, 255]
color2 = [0, 255, 0]
color3 = [255, 0, 0]
color4 = [0, 0, 0]
color5 = [255, 255, 255]
color = [color1, color2, color3, color4, color5]
```

```
for i in range(len(points_list)):
```

```
    points_i = points_list[i]
```

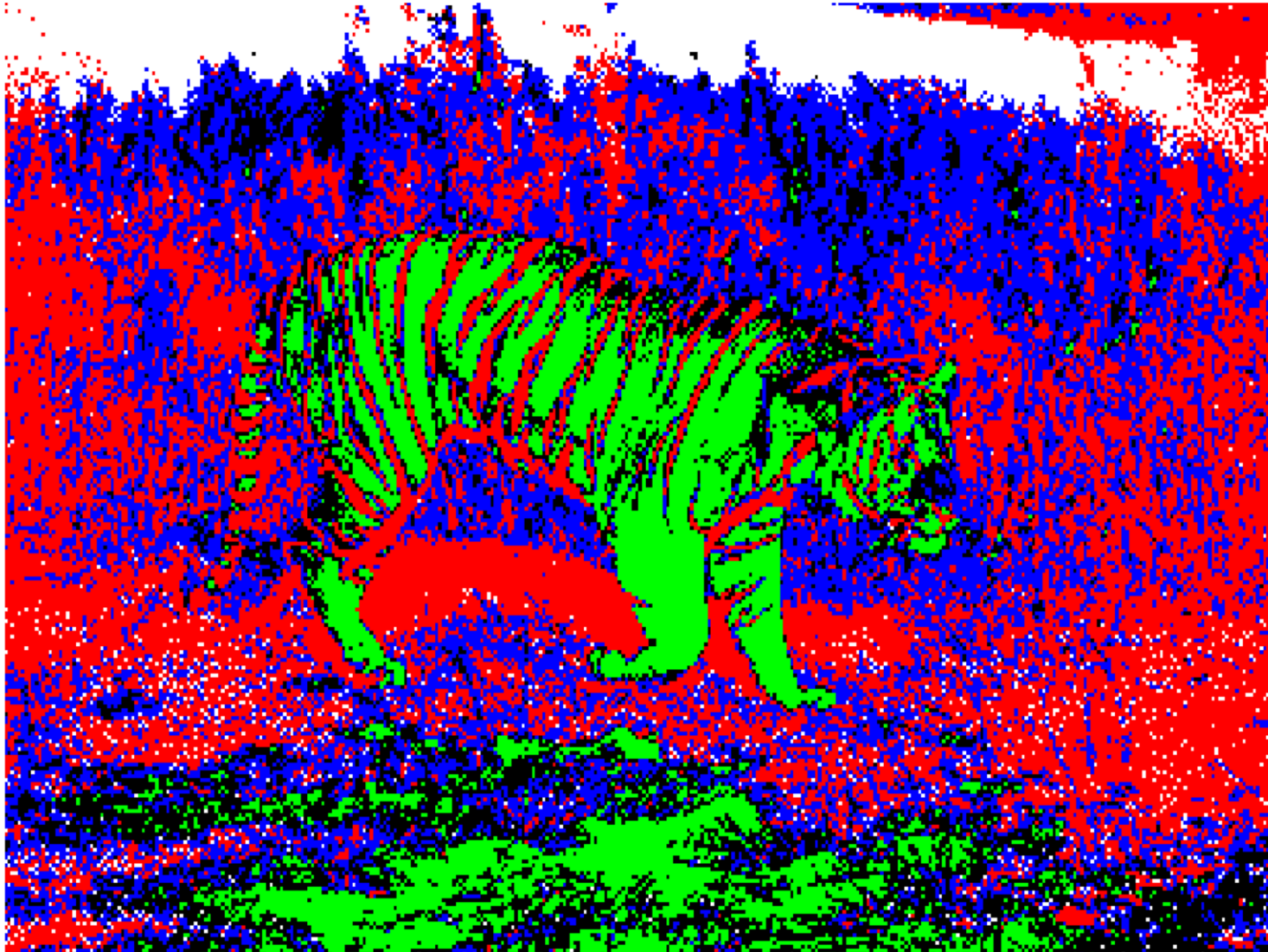
```
    img2[data1[points_i, 0],
```

```
        data1[points_i, 1], :] = color[i]
```

Gán màu cho
từng cụm dữ liệu

Lecture 5: Programming exercise

Kết quả phân cụm K-means



Lecture 5: Programming exercise

Bài 4: Cho ảnh như hình dưới đây (ảnh 'C5_B4.jpg' trong thư mục data đi kèm).



Hãy tiến hành phân đoạn ảnh trên với ngưỡng là 150. Áp dụng các toán tử dilation, erosion, opening, closing cho kết quả phân đoạn nói trên.

Lecture 5: Programming exercise

Phân ngưỡng ảnh

```
img = cv2.imread('./data/C5_B4.jpg')  
img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
img1 = img_gray.copy()
```

```
threshold = 150
```

```
img1[img_gray < threshold] = 255
```

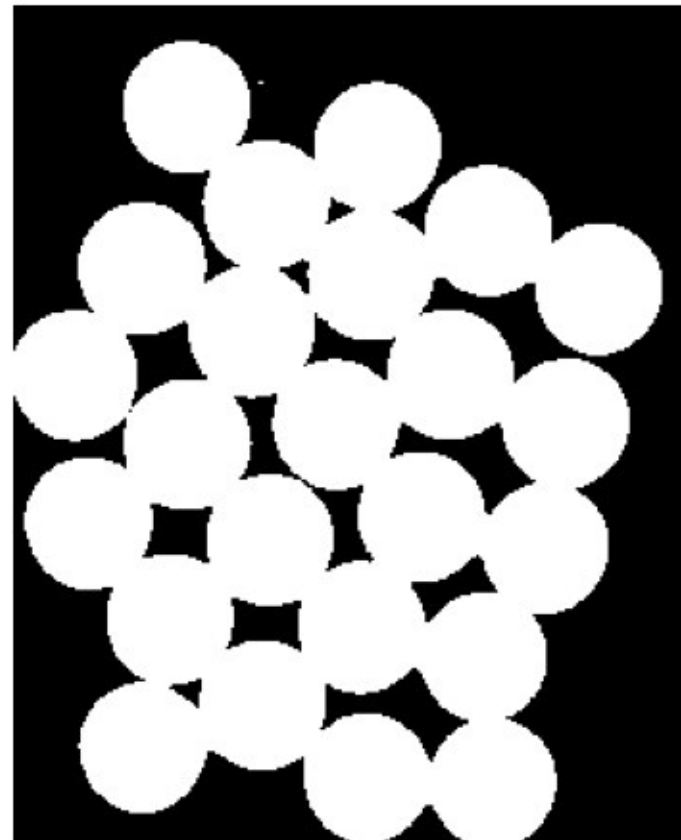
---> Các pixel có ngưỡng xám thấp sẽ cho thành pixel trắng (đồng xu)

```
img1[img_gray >= threshold] = 0
```

---> Các pixel có ngưỡng cao thấp sẽ cho thành pixel đen (nền)

Lecture 5: Programming exercise

Phân ngưỡng ảnh



Lecture 5: Programming exercise

Toán tử dilation

$$G = H \oplus F$$

Trong đó: H - ảnh đầu vào, G - ảnh đầu ra, F - bộ lọc

Quá trình tính toán:

- Di chuyển bộ lọc F đến từng vị trí của ảnh đầu vào
- Nếu trong vùng chồng lấn có một vị trí là mức 1 thì kết quả vị trí đó là 1, ngược lại nếu không có vị trí chồng lấn nào bằng 1 thì kết quả trả về bằng 0.

Lecture 5: Programming exercise

Toán tử dilation

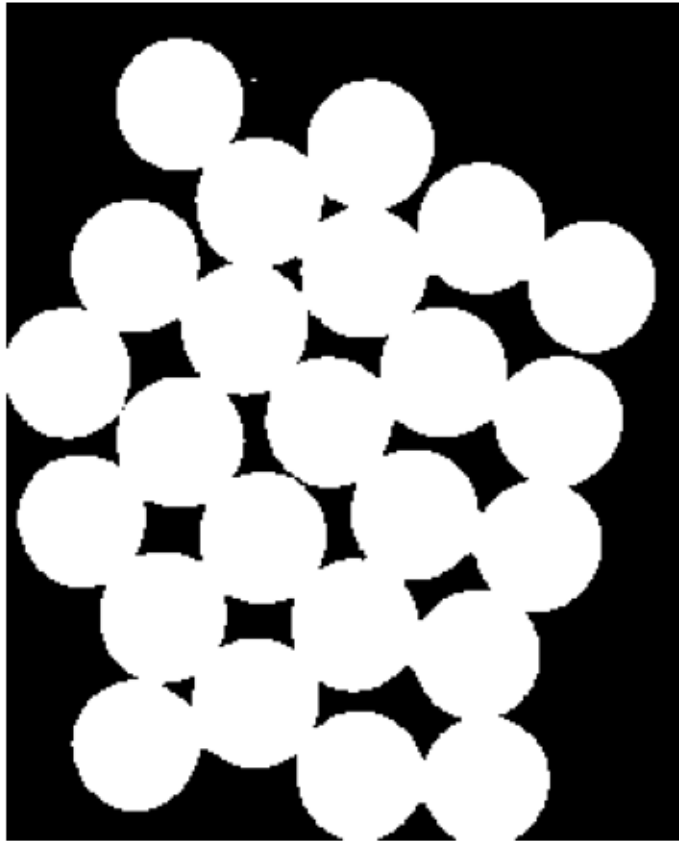
```
def DilationOperation(img, kernel_size):  
    K = np.ones((kernel_size, kernel_size),  
                dtype=np.int8)  
    temp = img.copy()  
  
    temp[temp == 255] = 1  
  
    img1 = cv2.filter2D(temp, -1, K,  
                       borderType=cv2.BORDER_CONSTANT)  
  
    img1[img1 != 0] = 255  
  
    return img1
```

Tính toán, kiểm tra các vị trí xung quanh thông qua phép tích chập

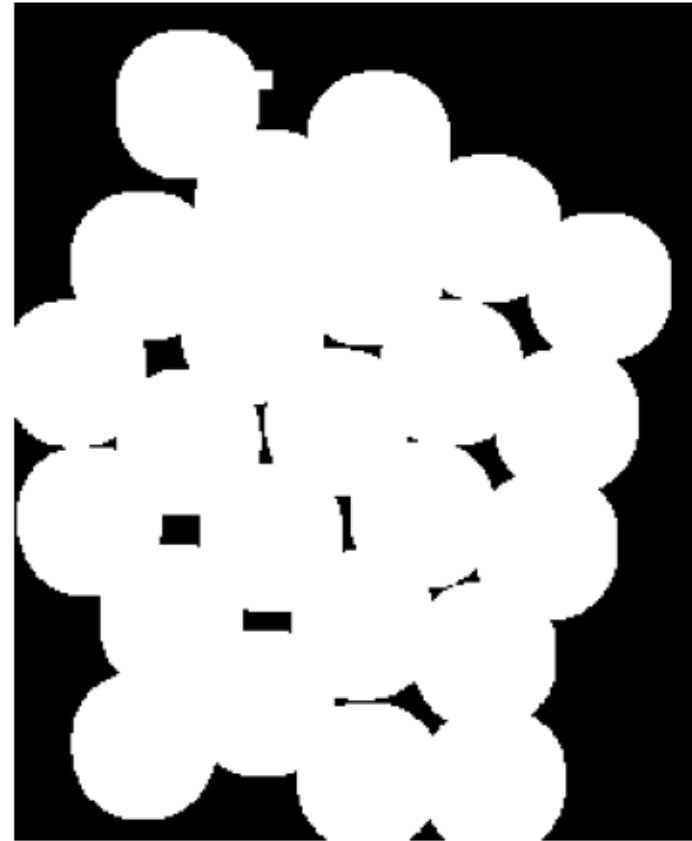
Lecture 5: Programming exercise

Toán tử dilation

Segmentation



Dilation



Lecture 5: Programming exercise

Toán tử erosion

$$G = H \ominus F$$

Trong đó: H - ảnh đầu vào, G - ảnh đầu ra, F - bộ lọc

Quá trình tính toán:

- Di chuyển bộ lọc F đến từng vị trí của ảnh đầu vào
- Nếu trong vùng chồng lấn tất cả các vị trí đều là mức 1 thì kết quả trả về là 1, ngược lại nếu có một vị trí bằng 0 thì kết quả trả về bằng 0.

Lecture 5: Programming exercise

Toán tử erosion

```
def ErosionOperation(img, kernel_size):
```

```
    K = np.ones((kernel_size, kernel_size),  
                dtype=np.int8)
```

```
    temp = img.copy()  
    temp[temp == 255] = 1
```

```
    img1 = cv2.filter2D(temp, -1, K,  
                        borderType=cv2.BORDER_CONSTANT)
```

```
    img1[img1 != kernel_size**2] = 0  
    img1[img1 == kernel_size**2] = 255
```

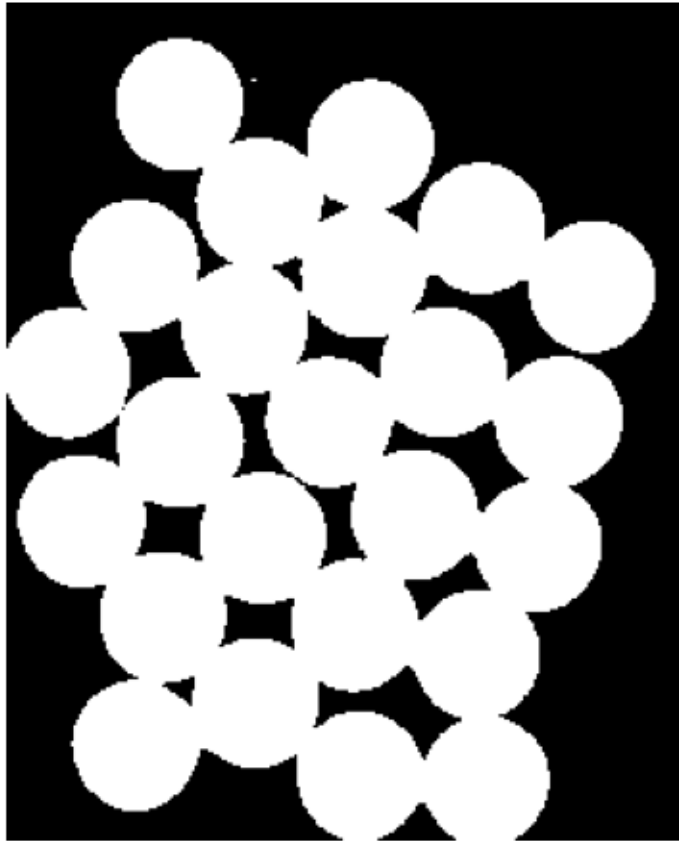
```
    return img1
```

Tính toán, kiểm tra các vị trí xung quanh thông qua phép tích chập

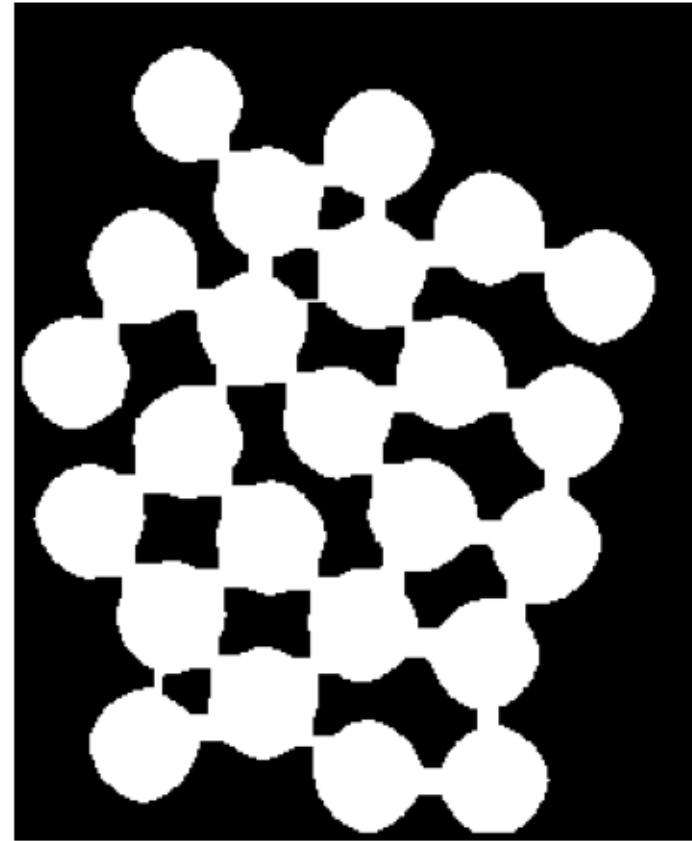
Lecture 5: Programming exercise

Toán tử erosion

Segmentation



Erosion



Lecture 5: Programming exercise

Toán tử opening

$$G = (H \ominus F) \oplus F$$

Trong đó: H - ảnh đầu vào, G - ảnh đầu ra, F – bộ lọc

Quá trình tính toán:

- Thực hiện với toán tử erosion trước
- Lấy kết quả đầu ra thực hiện với toán tử dilation

Lecture 5: Programming exercise

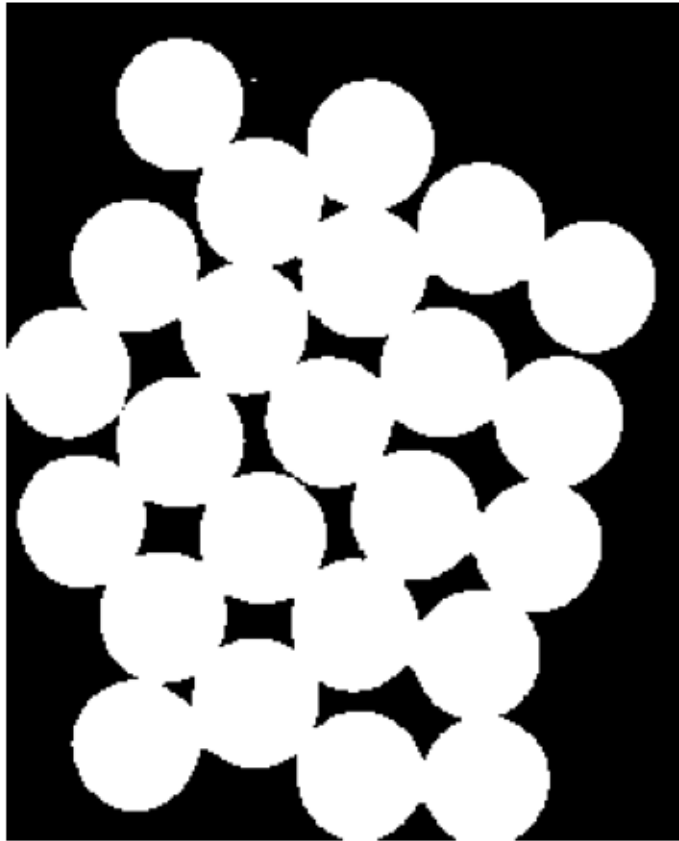
Toán tử opening

```
def OpeningOperation(img, kernel_size):  
    img1 = ErosionOperation(img, kernel_size)  
  
    img2 = DilationOperation(img1, kernel_size)  
  
    return img2
```

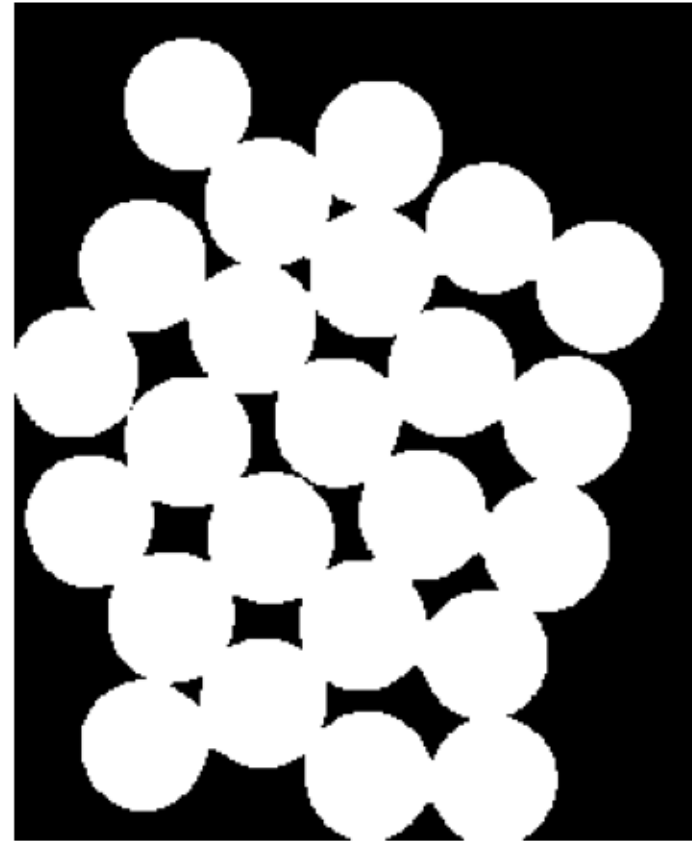
Lecture 5: Programming exercise

Toán tử opening

Segmentation



Opening



Lecture 5: Programming exercise

Toán tử closing

$$G = (H \oplus F) \ominus F$$

Trong đó: H - ảnh đầu vào, G - ảnh đầu ra, F – bộ lọc

Quá trình tính toán:

- Thực hiện với toán tử dilation trước.
- Lấy kết quả đầu ra thực hiện với toán tử erosion.

Lecture 5: Programming exercise

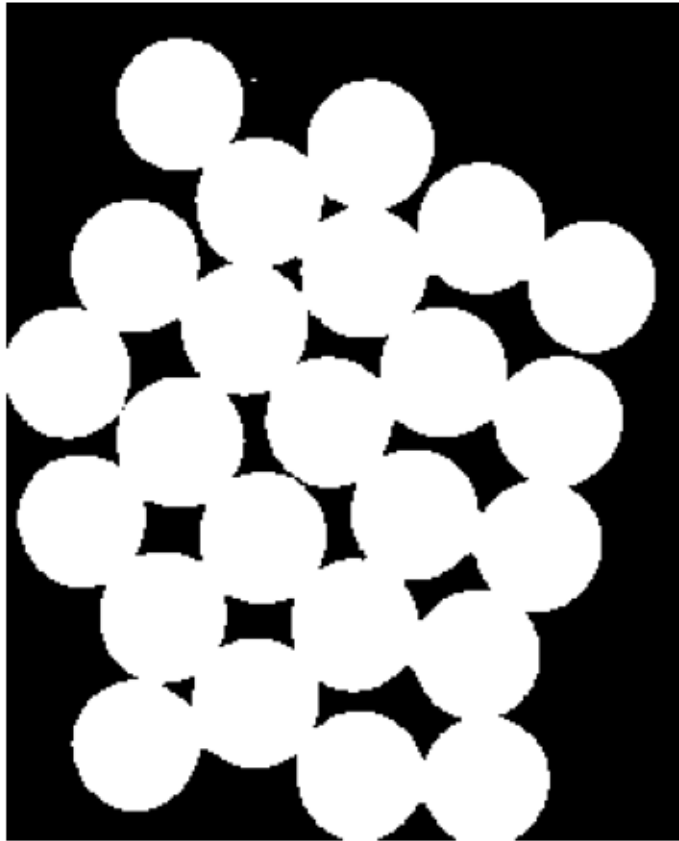
Toán tử closing

```
def ClosingOperation(img, kernel_size):  
    img1 = DilationOperation(img, kernel_size)  
  
    img2 = ErosionOperation(img1, kernel_size)  
  
    return img2
```

Lecture 5: Programming exercise

Toán tử closing

Segmentation



Closing

